

Input/Output Organization

Objective

This chapter discusses the techniques that computers use to communicate with input and output devices. Interface units are presented to show the way that the processor interacts with external peripherals. Four modes of transfer are discussed: programmed I/O, interrupt initiated transfer, direct memory access, and the use of input-output processors.

OUTLINE

- 9.1 Peripheral Devices
- 9.2 Input-Output Interface
- 9.3 Modes of Transfer
- 9.4 Direct Memory Access
- 9.5 Input-Output Processor
- 9.6 Serial Communication

9.1 INTRODUCTION TO PERIPHERAL DEVICES

The input-output subsystem of a computer, referred to as I/O, provides an efficient mode of communication between the central system and the outside environment. Programs and data must be entered into computer memory for processing and results obtained from computations must be recorded or displayed for the user. A computer serves no useful purpose without the ability to receive information from an outside source and to transmit results in a meaningful form. Any input/output devices connected to the computer are called peripheral devices.

9.2 INPUT-OUTPUT INTERFACE

Provides a method for transferring information between internal storage (such as memory and CPU registers) and external I/O devices, resolves the differences between the computer and peripheral devices

- Peripherals - Electromechanical Devices, CPU or Memory - Electronic Device
- Data Transfer Rate
 - ▲ Peripherals - Usually slower
 - ▲ CPU or Memory - Usually faster than peripherals
 - ▲ Some kinds of Synchronization mechanism may be needed
- Unit of Information
 - ▲ Peripherals - Byte, Block, ...
 - ▲ CPU or Memory - Word
- Data representations may differ

I/O Bus and Interface Modules

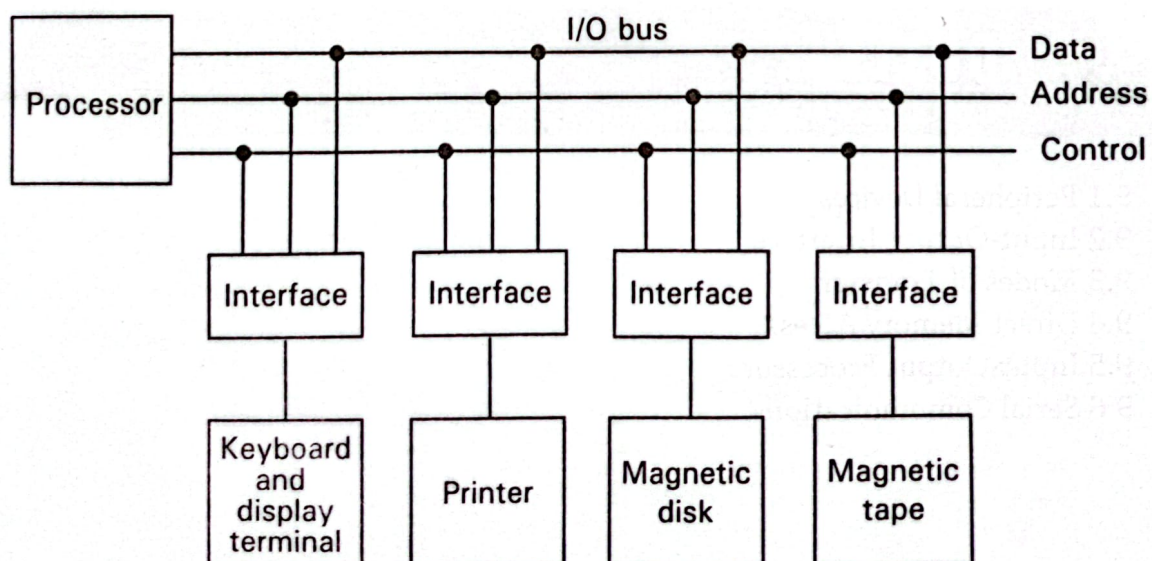


Fig 9.1: Connection of I/O bus to input-output devices.

I/O bus from the processor is connected to all peripheral interfaces. To communicate with a particular device, the processor places a device address on the address lines. Each peripheral has an interface module associated with its interface. Functions of an interface are as below:

- Decodes the device address (device code)
- Decodes the I/O commands (operation or function code)
- Provides signals for the peripheral controller
- Synchronizes the data flow and
- Supervises the transfer rate between peripheral and CPU or Memory

Types of I/O command

Control command:-Issued to activate the peripheral and to inform it what to do?

Status command:-Used to check the various status conditions of the interface before a transfer is initiated

Data input command:-Cause the interface to read the data from the peripheral and place it into the interface buffer.

Data output command:-Causes the interface to read the data from the bus and save it into the interface buffer

I/O bus and memory bus

Memory bus is used for information transfers between CPU and the MM (main memory). I/O bus is for information transfers between CPU and I/O devices through their I/O interface

Physical Organizations

Many computers use a common single bus system for both memory and I/O interface units

- Use one common bus but separate control lines for each function
- Use one common bus with common control lines for both functions
- Some computer systems use two separate buses,
- One to communicate with memory and the other with I/O interfaces

I/O Bus

Communication between CPU and all interface units is via a common I/O bus. An interface connected to a peripheral device may have a number of data registers, a control register, and a status register. A command is passed to the peripheral by sending to the appropriate interface register

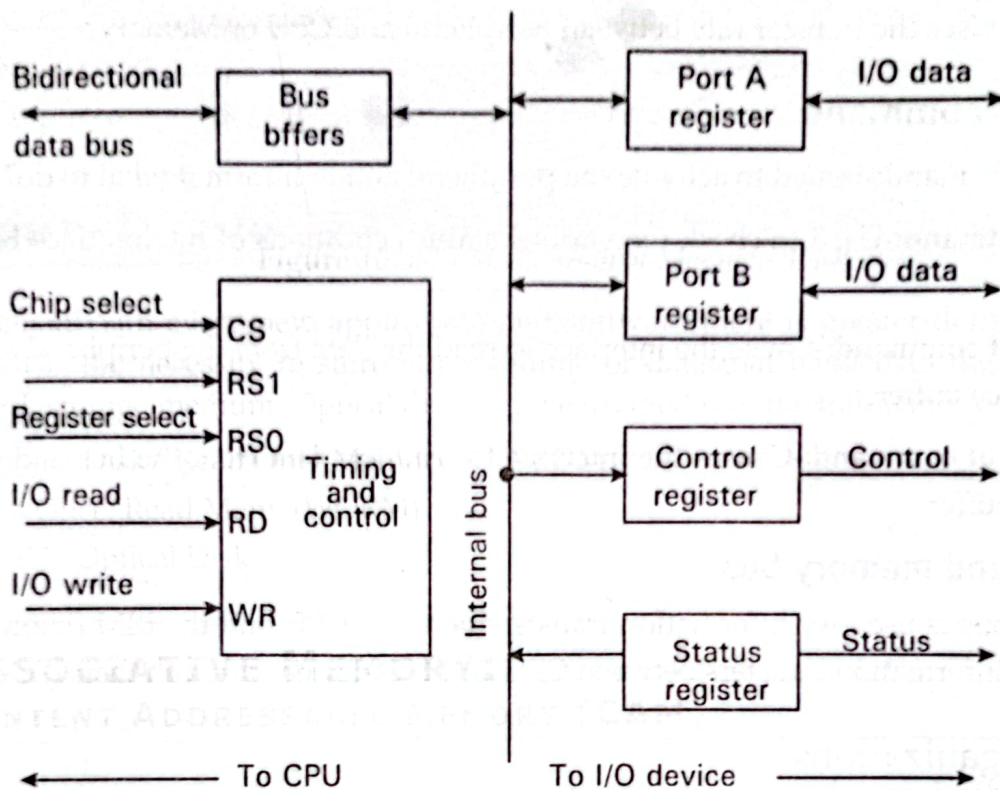
Isolated vs. Memory mapped I/O

Isolated I/O

- Separate I/O read/write control lines in addition to memory read/write control lines
- Separate (isolated) memory and I/O address spaces
- Distinct input and output instructions

Memory-mapped I/O

- A single set of read/write control lines (no distinction between memory and I/O transfer)
- Memory and I/O addresses share the common address space
 - ▲ reduces memory address range available
- No specific input or output instruction
 - ▲ The same memory reference instructions can be used for I/O transfers
- Considerable flexibility in handling I/O operations



CS	RS1	RS0	Register selected
0	x	x	None: data bus in high-impedance
1	0	0	Port A register
1	0	1	Port B register
1	1	0	Control register
1	0	1	Status register

Figure 9.2: Example of I/O interface unit

Interface communicates with the CPU through the data bus. The chip select and register select inputs determine the address assigned to the interface. Control lines I/O read and write are used to specify the input and output respectively. Bidirectional lines represent both data in and out from the CPU. Information in each port can be assigned a meaning depending on the mode of operation of the I/O device: Port A = Data; Port B = Command; Port C = Status. CPU initializes (loads) each port by transferring a byte to the Control Register. CPU can define the mode of operation of each port.

9.3 MODES OF TRANSFER

- Program-Controlled I/O
- Interrupt-Initiated I/O
- Direct Memory Access (DMA)

1. Program-Controlled I/O (Input Device to CPU)

- Continuous CPU involvement
- CPU slowed down to I/O speed
- Simple
- Least hardware

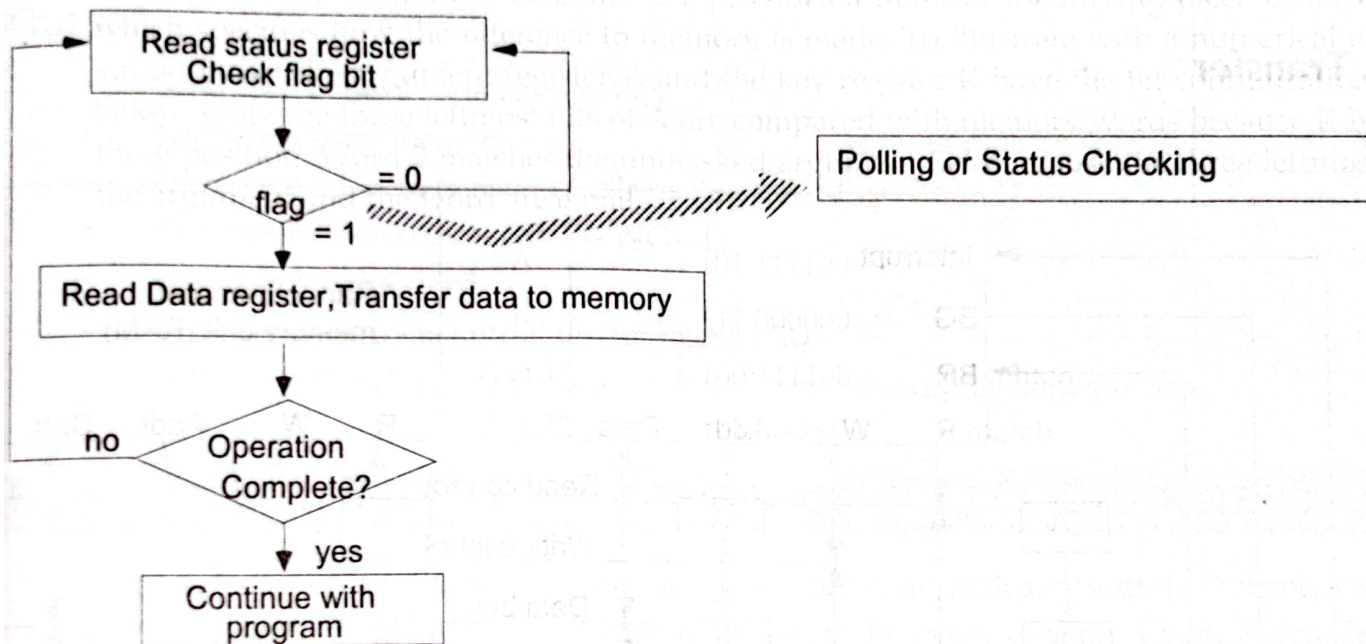


Figure 9.3: Flowchart of program-controlled IO Communication

2. Interrupt Initiated I/O

- Polling takes valuable CPU time
- Open communication only when some data has to be passed -> *Interrupt*.
- I/O interface, instead of the CPU, monitors the I/O device
- When the interface determines that the I/O device is ready for data transfer, it generates an *Interrupt Request* to the CPU
- Upon detecting an interrupt, CPU stops momentarily the task it is doing, branches to the service routine to process the data transfer, and then returns to the task it was performing

3. DMA (Direct Memory Access)

Large blocks of data transferred at a high speed to or from high speed devices, magnetic drums, disks, tapes, etc. DMA controller (Interface) provides I/O transfer of data directly to and from the memory and the I/O device. CPU initializes the DMA controller by sending a memory address and the number of words to be transferred. Actual transfer of data is done directly

between the device and memory through DMA controller freeing CPU for other tasks. DMA works by stealing the CPU cycles.

Cycle Stealing

While DMA I/O takes place, CPU is also executing instructions. DMA Controller and CPU both access Memory which causes memory Access Conflict. Memory Bus Controller is responsible for coordinating the activities of all devices requesting memory access by using priority schemes. Memory accesses by CPU and DMA Controller are interwoven; with the top priority given to DMA Controller which is called cycle Stealing. CPU is usually much faster than I/O(DMA), thus CPU uses the most of the memory cycles. DMA Controller steals the memory cycles from CPU for those stolen cycles, CPU remains idle. For those slow CPU, DMA Controller may steal most of the memory cycles which may cause CPU remain idle long time.

DMA Transfer

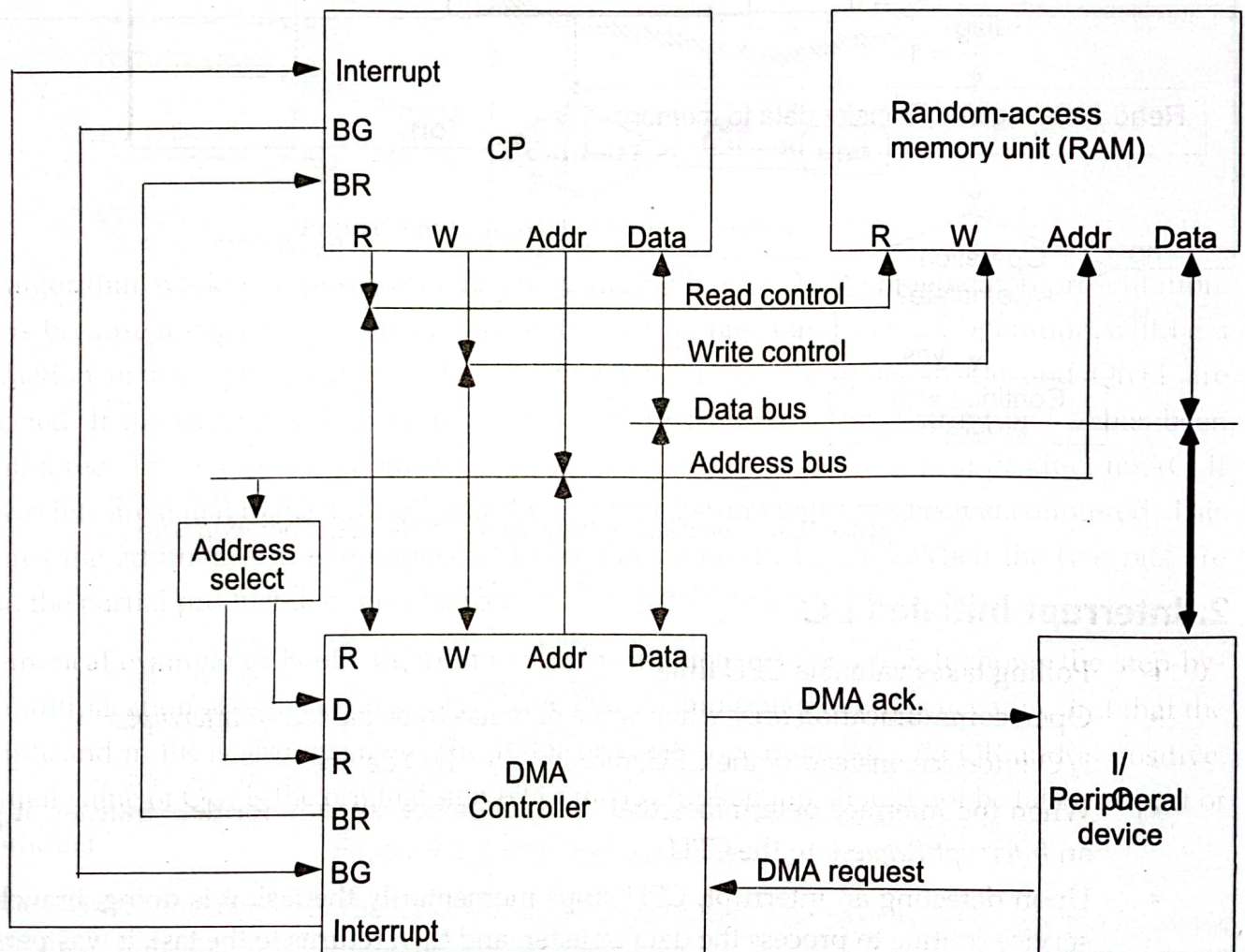


Figure 9.4: DMA communication block diagram.

CPU executes instruction to

- Load Memory Address Register
- Load Word Counter
- Load Function(Read or Write) to be performed
- Issue a GO command

Upon receiving a GO Command DMA performs I/O operation as follows independently from CPU

Input

- Send read control signal to Input Device
- DMA controller collects the input from input device byte by byte and assembles the byte into a word until word is full
- Send write control signal to memory
- Increment address register ($\text{Address Reg} \leftarrow \text{Address Reg} + 1$)
- Decrement word count ($\text{WC} \leftarrow \text{WC} - 1$)
- If $\text{WC} = 0$, then Interrupt to acknowledge done, else repeat same process

Output

- Send read control signal to memory
- Read data from memory
- Increment address register ($\text{Address Reg} \leftarrow \text{Address Reg} + 1$)
- Decrement word count ($\text{WC} \leftarrow \text{WC} - 1$)
- Disassemble the word
- Transfer data to the output device byte by byte
- If $\text{WC} = 0$, then Interrupt to acknowledge done, else repeat same process

9.4 INPUT OUTPUT PROCESSOR (I/O CHANNEL)

Processor with direct memory access capability that communicates with I/O devices is called I/O processor (channel). Channel accesses memory by cycle stealing. Channel can execute a channel program stored in the main memory. CPU initiates the channel by executing a channel I/O class instruction and once initiated, channel operates independently of the CPU.

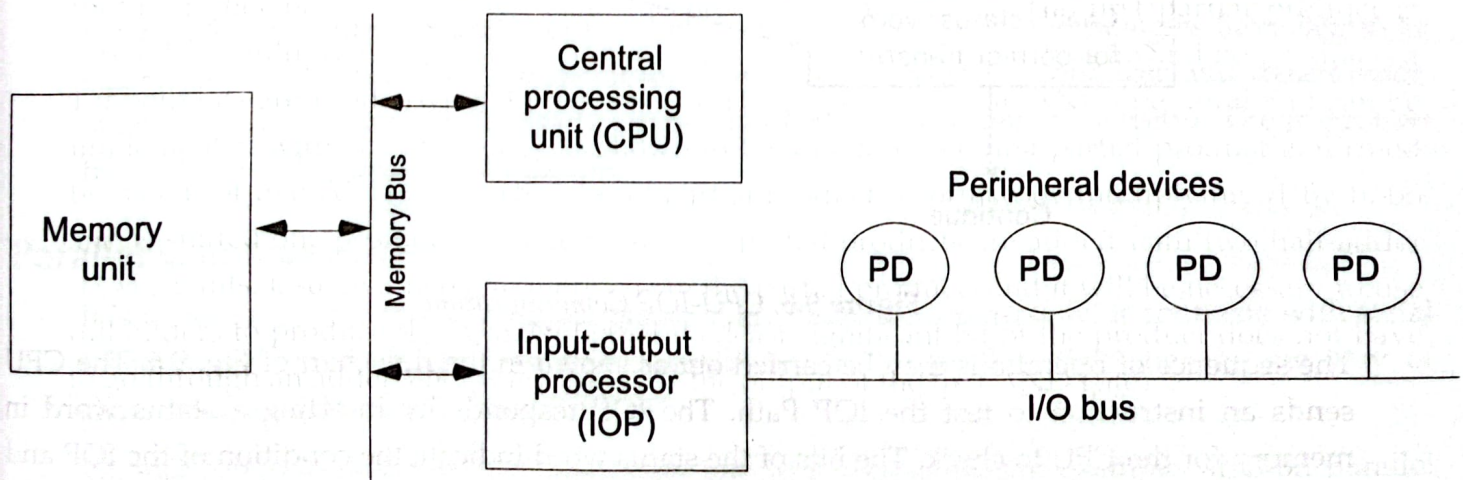


Figure 9.5: Block diagram of computer with IO processor

CPU-IOP Communication

The communication between CPU and IOP may take different forms, depending on the particular computer considered. In most cases the memory unit acts as a message center where

each processor leaves information for the other. To appreciate the operation of a typical IOP, we will illustrate by a specific example the method by which the CPU and IOP communicate. This is a simplified example that omits many operating details in order to provide an overview of basic concepts.

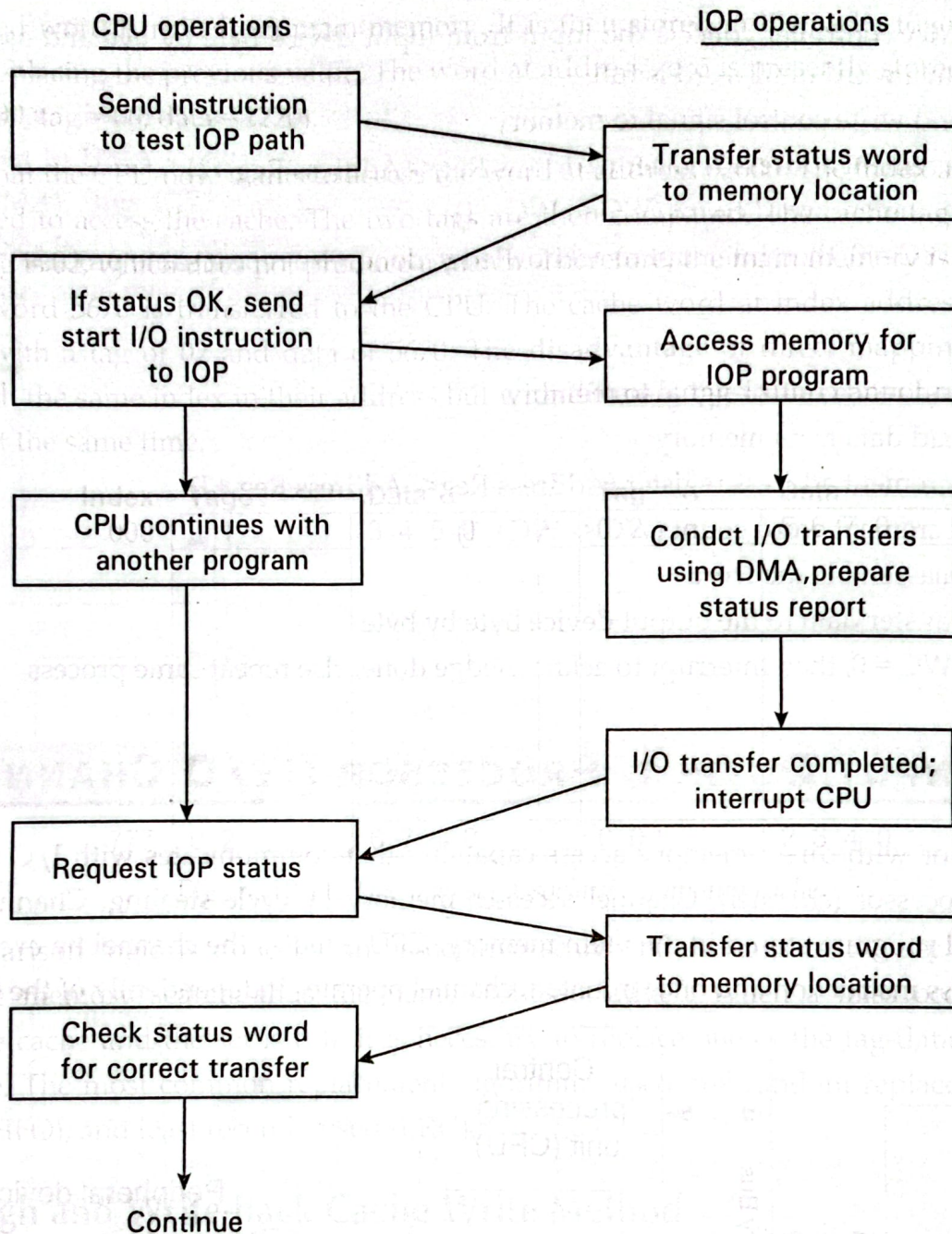


Figure 9.6: CPU-IOP Communication

The sequence of operations may be carried out as shown in the flowchart of Fig. 9.6. The CPU sends an instruction to test the IOP Path. The IOP responds by inserting a status word in memory for the CPU to check. The bits of the status word indicate the condition of the IOP and I/O devices, such as IOP overload condition, device busy with another transfer, or device ready for I/O transfer. The CPU refers to the status word in memory to decide what to do next. If all is in order, the CPU sends the instruction to start I/O transfer. The memory address received with this instruction tells the IOP where to find its program.

The CPU can now continue with another program while the IOP is busy with the I/O program. Both programs refer to memory by means of DMA transfer. When the IOP terminates the execution of its program, it sends an interrupt request to the CPU. The CPU responds to the interrupt by issuing an instruction to read the status from the IOP. The IOP responds by placing the contents of its status report into a specified memory location. The status word indicates whether the transfer has been completed or if any errors occurred during the transfer. From inspection of the bits in the status word, the CPU determines if the I/O operation was completed satisfactorily without errors.

The IOP takes care of all data transfers between several I/O units and the memory while the CPU is processing another program. The IOP and CPU are competing for the use of memory, so the number of devices that can be in operation is limited by the access time of the memory. It is not possible to saturate the memory by I/O devices in most systems, as the speed of most devices is much slower than the CPU. However, some very fast units, such as magnetic disks can use an appreciable number of the available memory cycles. In that case, the speed of the CPU may deteriorate because it will often have to wait for the IOP to conduct memory transfers.

9.5 SERIAL AND PARALLEL COMMUNICATION

Serial Communication

It is the process of sending data one bit at a time, sequentially, over a communication channel or computer bus. This is in contrast to parallel communication, where several bits are sent as a whole, on a link with several parallel channels.

Serial communication is used for all long-haul communication and most computer networks, where the cost of cable and synchronization difficulties makes parallel communication impractical. Serial computer buses are becoming more common even at shorter distances, as improved signal integrity and transmission speeds in newer serial technologies have begun to outweigh the parallel bus's advantage of simplicity (no need for serializer and deserializer, or SerDes) and to outstrip its disadvantages (clock skew, interconnect density). The migration from PCI to PCI Express is an example.

Parallel Communication

It is a method of conveying multiple binary digits (bits) simultaneously. It contrasts with serial communication, which conveys only a single bit at a time; this distinction is one way of characterizing a communications link.

Parallel communication implies more than one such conductor. For example, an 8-bit parallel channel will convey eight bits (or a byte) simultaneously, whereas a serial channel would convey those same bits sequentially, one at a time. If both channels operated at the same clock speed, the parallel channel would be eight times faster. A parallel channel may have additional conductors for other signals, such as a clock signal to pace the flow of data, a signal to control the direction of data flow, and handshaking signals.

Data Communication Processor

A data communication Processor is an I/O processor that distributes and collects data from many remote terminals connected through telephone and other communication lines. It is a specialized I/O processor designed to communicate directly with data communication networks. A communication network may consist of any of a wide variety of devices, such as printers, interactive display devices, digital sensors, or a remote computing facility. With the use of a data communication processor, the computer can service fragments of each network demand in an interspersed manner and thus have the apparent behavior of serving many users at once. In this way the computer is able to operate efficiently in a time-sharing environment.

The most striking difference between an I/O processor and a data communication processor is in the way the processor communicates with the I/O devices. An I/O processor communicates with the Peripherals through a common I/O bus that is comprised of many data and control lines. All peripherals share the common bus and use it to transfer information to and from the I/O processor. A data communication processor communicates with each terminal through a single pair of wires. Both data and control information are trans-character received. Another procedure used in asynchronous terminals involving a human operator is to *echo* the character. The character transmitted from the keyboard to the computer is recognized by the processor and retransmitted to the terminal printer. The operator would realize that an error occurred during transmission if the character printed is not the same as the character whose key he has struck.

Modes of Data Transfer

Data can be transmitted between two points in three different modes: simplex, half-duplex, or full-duplex.

Simplex

A simplex line carries information in one direction only. This mode is seldom used in data communication because the receiver cannot communicate with the transmitter to indicate the occurrence of errors. Examples of simplex transmission are radio and television broadcasting.

Half-Duplex

A half-duplex transmission system is one that is capable of transmitting in both directions but data can be transmitted in only one direction at a time. A pair of wires is needed for this mode. A common situation is for one modem to act as the transmitter and the other as the receiver. When transmission in one direction is completed, the role of the modems is reversed to enable transmission in the reverse direction. The time required to switch a half-duplex line from one direction to the other is called the turnaround time.

Full-Duplex

A full-duplex transmission can send and receive data in both directions simultaneously. This can be achieved by means of a four-wire link, with a different pair of wires dedicated to each direction of transmission. Alternatively, a two-wire circuit can support full-duplex communication if the frequency spectrum is subdivided into two non-overlapping frequency bands to create separate receive and transmit channels in the same physical pair of wires.

Protocol

The communication lines, modems, and other equipment used in the transmission of information between two or more stations are called a data link. The orderly transfer of information in a data link is accomplished by means of a protocol. A data link control protocol is a set of rules that are followed by interconnecting computers and terminals to ensure the orderly transfer offered in a serial fashion with the result that the transfer rate is much slower. The task of the data communication processor is to transmit and collect digital information to and from each terminal, determine if the information is data or control and respond to all requests according to predetermined established procedures. The processor, obviously, must also communicate with the CPU and memory in the same manner as any I/O processor.

Character-Oriented Protocols

In a character-oriented protocol, data to be carded are 8-bit characters from a coding system. The header, which normally carries the source and destination addresses and other control information, and the trailer, which carries error detection or error correction redundant bits, are also multiples of 8 bits. To separate one frame from the next, an 8-bit (1-byte) flag is added at the beginning and the end of a frame. The flag, composed of protocol-dependent special characters, signals the start or end of a frame. Figure 1 shows the format of a frame in a character-oriented protocol.

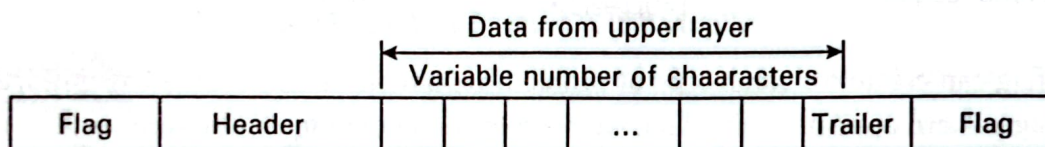


Figure 9.7: A frame in a character-oriented protocol

Character-oriented framing was popular when only text was exchanged by the data link layers. The flag could be selected to be any character not used for text communication. Now, however, we send other types of information such as graphs, audio, and video. Any pattern used for the flag could also be part of the information. If this happens, the receiver, when it encounters this pattern in the middle of the data, thinks it has reached the end of the frame. To fix this problem, a byte-stuffing strategy was added to character-oriented framing. In byte stuffing (or character stuffing), a special byte is added to the data section of the frame when there is a character with the same pattern as the flag. The data section is stuffed with an extra byte. This byte is usually called the escape character (ESC), which has a predefined bit pattern. Whenever the receiver encounters the ESC character, it removes it from the data section and treats the next character as data, not a delimiting flag.

Byte stuffing by the escape character allows the presence of the flag in the data section of the frame, but it creates another problem. What happens if the text contains one or more escape characters followed by a flag? The receiver removes the escape character, but keeps the flag,

which is incorrectly interpreted as the end of the frame. To solve this problem, the escape characters that are part of the text must also be marked by another escape character. In other words, if the escape character is part of the text, an extra one is added to show that the second one is part of the text.

Character-oriented protocols present another problem in data communications. The universal coding systems in use today have 16-bit and 32-bit characters that conflict with 8-bit characters. We can say that in general, the target is moving toward the bit-oriented protocols that we discuss next.

Bit-Oriented Protocols

In a bit-oriented protocol, the data section of a frame is a sequence of bits to be interpreted by the upper layer as text, graphic, audio, video, and so on. However, in addition to headers (and possible trailers), we still need a delimiter to separate one frame from the other. Most protocols use a special 8-bit pattern flag 01111110 as the delimiter to define the beginning and the end of the frame, as shown in Figure 2.

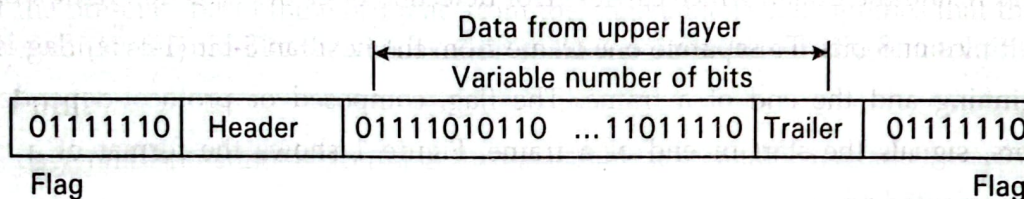


Figure 9.8: A frame in a bit-oriented protocol

This flag can create the same type of problem we saw in the byte-oriented protocols. That is, if the flag pattern appears in the data, we need to somehow inform the receiver that this is not the end of the frame. We do this by stuffing 1 single bit (instead of 1 byte) to prevent the pattern from looking like a flag. The strategy is called bit stuffing. In bit stuffing, if a 0 and five consecutive 1 bits are encountered, an extra 0 is added. This extra stuffed bit is eventually removed from the data by the receiver. Note that the extra bit is added after one 0 followed by five 1s regardless of the value of the next bit. This guarantees that the flag field sequence does not inadvertently appear in the frame.

Bit stuffing is the process of adding one extra 0 whenever five consecutive 1s follow a 0 in the data, so that the receiver does not mistake the pattern 01111110 for a flag.

Figure 3 shows bit stuffing at the sender and bit removal at the receiver. Note that even if we have a 0 after five 1s, we still stuff a 0. The 0 will be removed by the receiver.

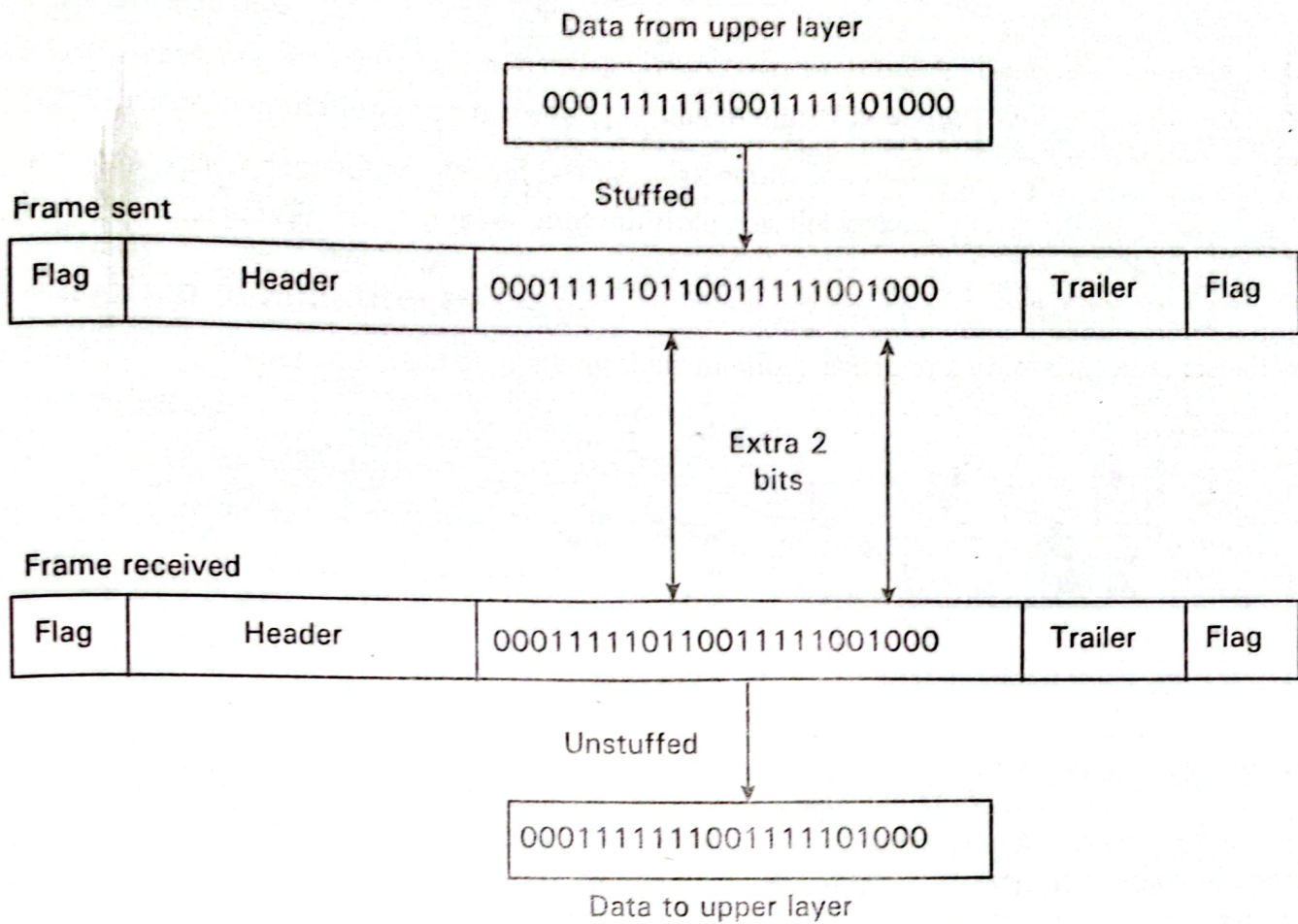


Figure 9.9: Bit stuffing and unstuffing

This means that if the flag like pattern 01111110 appears in the data, it will change to 011111010 (stuffed) and is not mistaken as a flag by the receiver. The real flag 01111110 is not stuffed by the sender and is recognized by the receiver.

